TIME COMPLEXITY

What is Time Complexity?

Time Complexity measures how the execution time of an algorithm grows as the input size (n) increases.

👉 In simple terms:

"It tells how fast or slow an algorithm scales when input size grows."

Example:

n = 10 → fast

n = 10,000 → may become slow

So we focus on the growth rate, not exact time.

Most Common Time Complexities (with Meaning)

Complexity	Meaning	Example
O(1)	Constant time	Array index access
O(n)	Linear growth	Single loop
O(log n)	Input halves each step	Binary Search
O(n log n)	Divide & process	Merge Sort
$O(n^2)$	Nested loops	Brute force pairs
$O(2^n)$	Exponential branching	Subsets
O(n!)	Factorial growth	Permutations

How to Calculate Time Complexity (Easy Method)

✔ Rule 1: Count loops

1 loop → O(n)

2 nested loops → $O(n^2)$

✔ Rule 2: Check loop growth

```
for(i = 1; i < n; i *= 2)
```

👉 O(log n) (because it doubles each step)

✔ Rule 3: Sequential loops add

```
for(i = 1 to n)
for(i = 1 to n)
```

👉 O(n + n) = O(n)

✔ Rule 4: Nested loops multiply

```
for i in n
  for j in n
```

👉 $O(n^2)$

✔ Rule 5: Ignore constants

O(2n + 5) → O(n)

✔ Rule 6: Ignore lower order terms

$O(n^2 + n) \rightarrow O(n^2)$

Pattern Recognition Tricks

Pattern	Complexity
i++	O(n)
i += 2	O(n)
i *= 2	O(log n)
i /= 2	O(log n)
Nested loops same size	$O(n^2)$
Nested loop + log	O(n log n)

Big O vs Big Ω vs Big Θ

Notation	Meaning
Big O (O)	Worst case
Big Omega (Ω)	Best case
Big Theta (Θ)	Average case

👉 In real-world and interviews, we mostly use Big O because we need to know the maximum possible time.

Real Code Examples

Example 1

```
for(i=1; i<=n; i++)
```

👉 O(n)

Example 2

```
for(i=1; i<=n; i++)
  for(j=1; j<=n; j++)
```

👉 $O(n^2)$

Example 3

```
for(i=1; i<=n; i++)
  for(j=1; j<=i; j++)
```

👉 $O(n^2/2) = O(n^2)$

Example 4

```
for(i=1; i<=n; i++)
  for(j=1; j<=log n; j++)
```

👉 O(n log n)

Example 5

```
while(n > 1)
  n = n / 2
```

👉 O(log n)

Example 6 (Recursion)
Fibonacci → $O(2^n)$

Interview Tricks (Very Important)
🔥 Trick 1: Identify brute force

👉 Nested loops usually mean brute force

🔥 Trick 2: Optimize using patterns

Replace nested loops with:

Two pointers

Sliding window

Hashing

Binary search

🔥 Trick 3: Sorted array?

👉 Think O(log n) or O(n) solution

🔥 Trick 4: Avoid repeated work

👉 Use HashMap / caching

Optimization Examples

From	To	Technique
$O(n^2)$	O(n)	Hashing
$O(n^2)$	O(n)	Two Pointers
O(n)	O(log n)	Binary Search
$O(2^n)$	O(n)	Dynamic Programming

Common Beginner Mistakes ❌

- Only counting loops
- Ignoring inner loop growth
- Including constants
- Writing best case as final answer
- Misunderstanding recursion

Quick Cheat Sheet ⚡

Pattern Complexity

Single loop	O(n)
Double loop	$O(n^2)$
Half each step	O(log n)
Divide & Conquer	O(n log n)
Subsets	$O(2^n)$
Permutations	O(n!)

Golden Rules

✔ Focus on growth, not exact time
✔ Always consider worst case
✔ Identify patterns
✔ Think optimization approaches
✔ Practice consistently

TIME COMPLEXITY – INTERVIEW FAQs

❓ 1. What is Time Complexity?

👉 Measures how runtime grows with input size
Interview line:

"It measures how execution time increases as input size increases."

❓ 2. Why Big O?

👉 Because it tells worst-case performance

❓ 3. Big O vs Ω vs Θ?

Notation	Meaning
O	Worst case
Ω	Best case
Θ	Average case

❓ 4. O(n log n) vs O(n^2)?

👉 For large inputs:
✔ O(n log n) is much faster

❓ 5. Is nested loop always O(n^2)?

❌ No — depends on inner loop

❓ 6. Steps to calculate complexity

Count loops
Check growth pattern
Multiply nested loops
Ignore constants

❓ 7. Space vs Time tradeoff

👉 Extra memory use karke time reduce karte hai

Example:
$O(n^2) \rightarrow O(n)$ using Hashing

❓ 8. How to reduce $O(n^2) \rightarrow O(n)$?

✔ Hashing
✔ Two pointers
✔ Sliding window
✔ Prefix sum

❓ 9. What is logarithmic complexity?

👉 Input har step me half hota hai
Example: Binary Search

❓ 10. Why Binary Search is O(log n)?

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \ldots$

❓ 11. Exponential complexity?

👉 Branching recursion
Example: Fibonacci, Subsets
👉 $O(2^n)$

❓ 12. Factorial complexity?

👉 All permutations
👉 $O(n!)$

❓ 13. Recursion time complexity depends on:

Number of calls
Depth of recursion

❓ 14. How to analyse recursion?

✔ Recursion Tree
✔ Master Theorem

❓ 15. Master Theorem (idea)

Used in divide & conquer:

$T(n) = aT(n/b) + f(n)$

Example: Merge Sort → O(n log n)

❓ 16. Why ignore constants?

👉 Growth matters, exact time nahi
O(2n + 5) → O(n)

❓ 17. Amortized complexity?

👉 Average cost per operation
Example: Dynamic array push → O(1)

❓ 18. Common operation complexities

Operation	**Complexity**
Array access	O(1)
Linear search	O(n)
Binary search	O(log n)
Merge sort	O(n log n)
Bubble sort	$O(n^2)$

❓ 19. Why O(n log n) optimal for sorting?

👉 Comparison-based sorting ka lower bound = O(n log n)

❓ 20. Common optimization patterns

✔ Two Pointers
✔ Sliding Window
✔ Binary Search
✔ Hashing
✔ Recursion → DP

❓ 21. When to use Two Pointers?

👉 Sorted arrays, pairs, triplets, target sum

❓ 22. When to use Sliding Window?

👉 Subarray / substring / continuous window problems

❓ 23. How to identify O(log n)?

👉 Look for:

i = i / 2

i = i * 2

Divide in half

❓ 24. Why recursion can be bad?

High time complexity
Stack overflow risk

❓ 25. How to optimize recursion?

✔ Memoization
✔ Tabulation (Dynamic Programming)